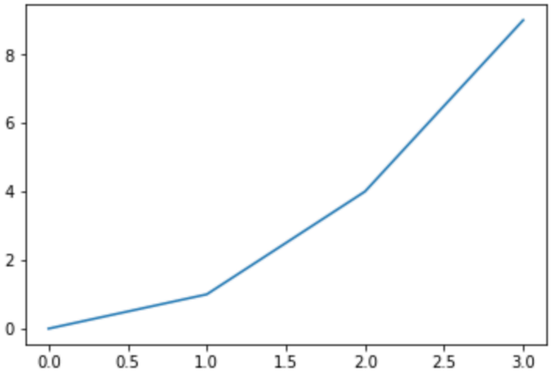
2D Plotting in Python

- **Overview**:

Python's matplotlib package is essential for creating visual plots. It's commonly used in scientific and engineering contexts for data visualization.

- **Basic Plotting Function**:
  - The function plot(x, y) creates a 2D plot from two lists or arrays, x and y, representing the coordinates.
  - Example:

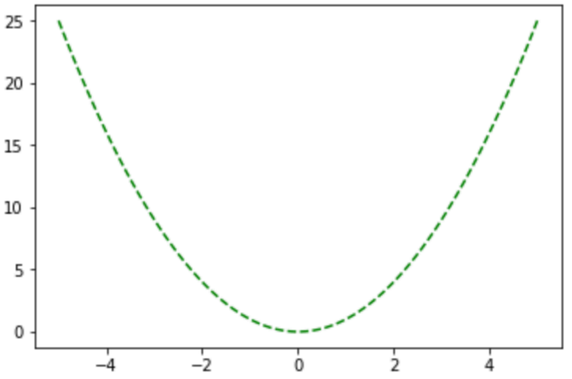
```python
x = [0, 1, 2, 3]
y = [0, 1, 4, 9]
plt.plot(x, y)
plt.show()
```



- **Interactive Features**:
  - In Jupyter Notebooks, %matplotlib notebook allows interactive features like panning and zooming.
  - %matplotlib inline disables these features for static images.

- **Customization**:
  - Line and marker styles can be customized, e.g., 'ro' plots red circles, while 'g--' creates a dashed green line.

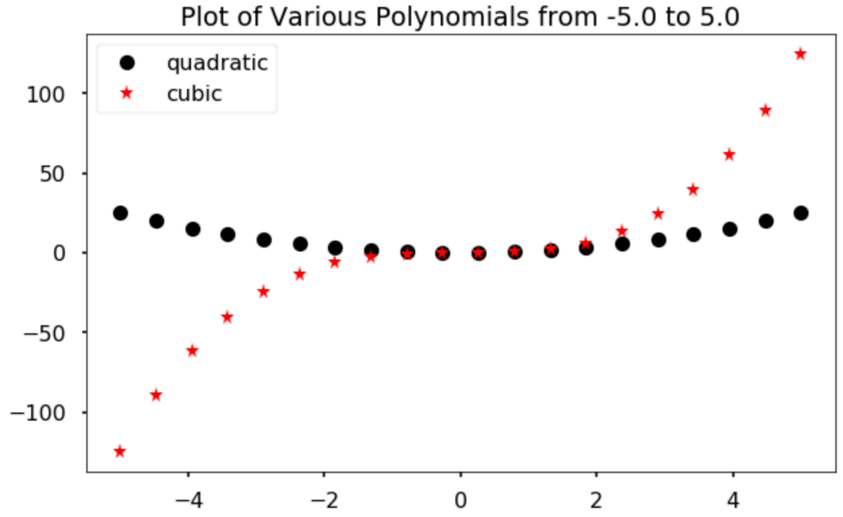
```python
x = np.linspace(-5,5,100)
plt.plot(x, x**2, 'g--')
plt.show()
```



Enhancing 2D Plots with Titles, Labels, and Legends

- **Adding Titles and Labels**:
  - Titles and axis labels provide context to the plot and can be customized for clarity.
  - Example:

```python
plt.title('Quadratic Function')
plt.xlabel('X axis')
plt.ylabel('Y axis')
```



- **Figure Size**:
  - Customizing the plot size can improve readability.

```python
plt.figure(figsize=(10, 6))
```

- **Legends and Styles**:
  - Multiple datasets can be plotted in one figure. Adding a label argument to plot() and using plt.legend() helps distinguish different datasets.
  - Styles, such as seaborn and ggplot, can be applied globally with plt.style.use('seaborn-poster').

Example of multiple plots with legends:

```python
x = np.linspace(-5,5,100)
plt.plot(x, x**2, 'ko', label='Quadratic')
plt.plot(x, x**3, 'r*', label='Cubic')
plt.legend(loc=2)
plt.show()
```

3D Plotting with Matplotlib

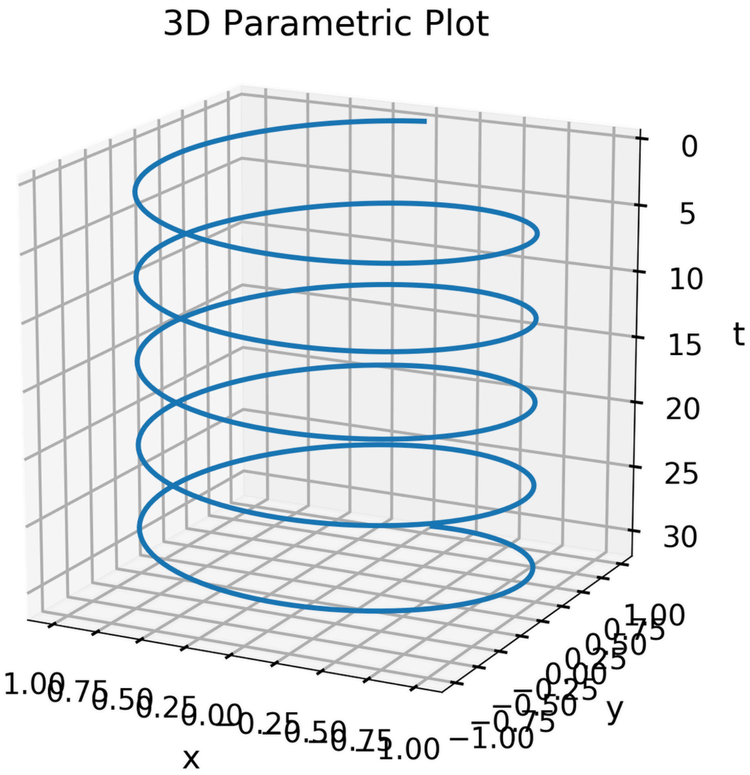
- **Matplotlib** is a powerful plotting library in Python. To plot 3D figures, the mplot3d toolkit must be imported and axes defined (ax = plt.axes(projection='3d') commonly).
- The 3D plot functionalities include:
  - Line plots using plot3D().
  - Scatter plots using scatter().
  - Surface plots using plot_surface().

Example 1: 3D Line Plot

- **Goal**: Create a 3D line plot using parametric data.
- **Key Code**:

```python
import numpy as np
from mpl_toolkits import mplot3d
import matplotlib.pyplot as plt

fig = plt.figure(figsize=(10,10))
ax = plt.axes(projection='3d')
t = np.arange(0, 10*np.pi, np.pi/50)
x = np.sin(t)
y = np.cos(t)
ax.plot3D(x, y, t)
ax.set_title('3D Parametric Plot')
plt.show()
```



- Result: A 3D parametric plot of (x, y, t) , with grid and axis labels enabled.

Surface and Scatter Plotting in 3D

3D Surface Plot:

- **Purpose**: To visualize functions of two variables Z = f(X, Y) , where X and Y are generated from np.meshgrid().
- Steps:

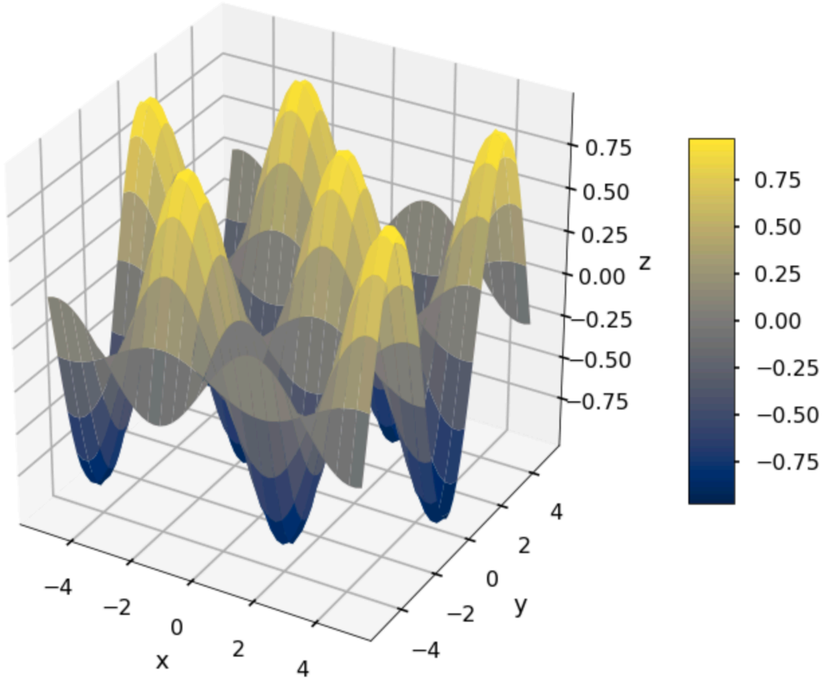
1.Create a meshgrid using lists/arrays for X and Y.

2.Compute Z values using the function Z = sin(X) * cos(Y) .

3.Plot the surface with plot_surface(), using a colormap (e.g., plt.cm.cividis).

- **Key Code:**

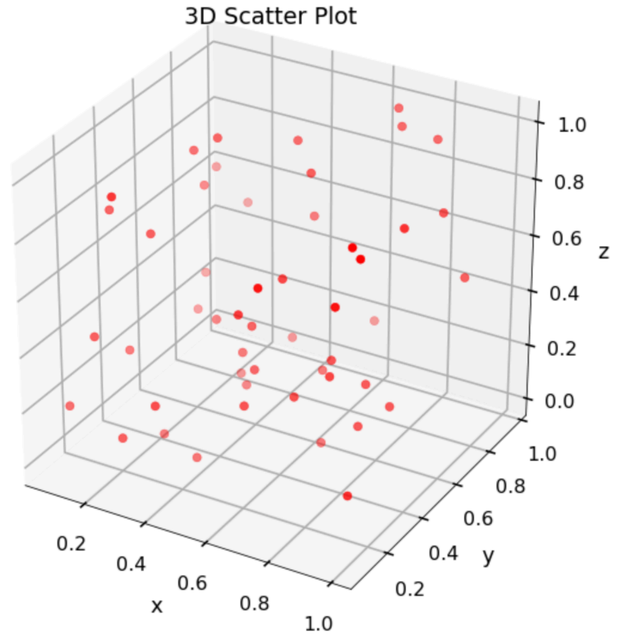
```python
x = np.arange(-5, 5.1, 0.2)
y = np.arange(-5, 5.1, 0.2)
X, Y = np.meshgrid(x, y)
Z = np.sin(X)*np.cos(Y)
ax.plot_surface(X, Y, Z, cmap=plt.cm.cividis)
plt.show()
```



3D Scatter Plot:

- **Goal**: Create a 3D scatter plot with randomly generated data.
- **Key Code**:

```python
x = np.random.random(50)
y = np.random.random(50)
z = np.random.random(50)
ax.scatter(x, y, z, c='r', s=50)
plt.show()
```



Working with Maps in Python

- **Purpose**: Engineers and scientists often need to interact with maps to visualize spatial data.

- **Python Packages for Maps**:
  - **Basemap**: Initially used with Matplotlib, deprecated after 2016.
  - **Cartopy**: Replaced Basemap, offers robust map projections and plotting functionalities.
  - **Folium**: Enables interactive web-based maps.

- **Basics of Mapping**:
  - A map is a **2D plot** with specific projections.
  - **Longitude**: Ranges from -180 to 180 degrees (East-West position).
  - **Latitude**: Ranges from -90 to 90 degrees (North-South position).

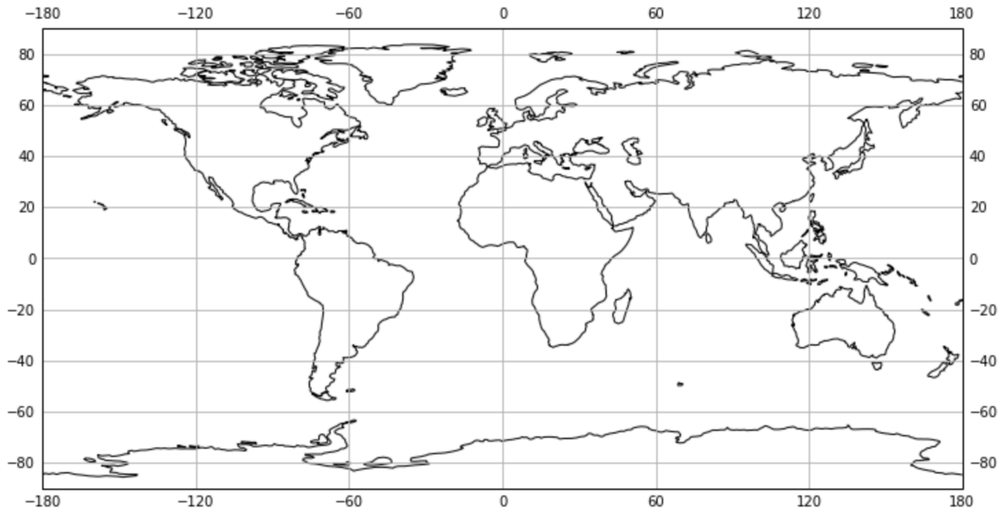
Plotting Maps with Cartopy

- **Map Projections**: Cartopy supports various projections, with **Plate Carrée** being a common choice.

- **Steps to Plot a Map**:
  - Use **Matplotlib** for creating plots.
  - Set map projection: ax = plt.axes(projection=ccrs.PlateCarree())
  - Add features like coastlines and gridlines.

```python
import cartopy.crs as ccrs
import matplotlib.pyplot as plt

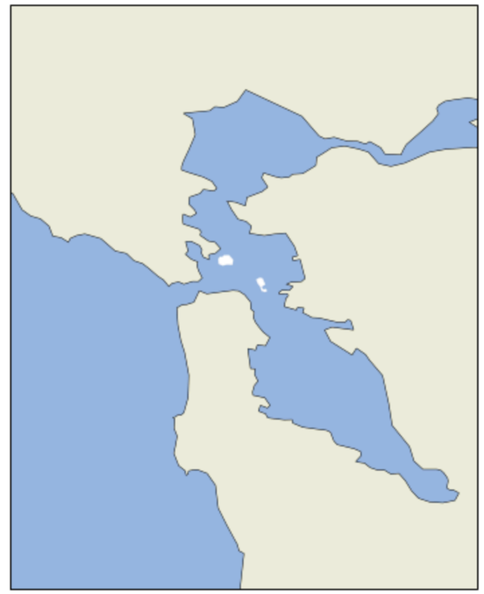
plt.figure(figsize=(12, 8))
ax = plt.axes(projection=ccrs.PlateCarree())
ax.coastlines()
ax.gridlines(draw_labels=True)
plt.show()
```



- **Enhancements**: Background maps can be improved using downsampled Natural Earth images.

Advanced Map Features in Cartopy

- **Adding Features**:
  - **ax.set_extent()**: Zoom into specific regions by specifying longitude and latitude limits.
  - Add **land, ocean, borders, lakes, rivers** using Cartopy's built-in features.



```python
import cartopy.feature as cfeature

ax.add_feature(cfeature.LAND)
ax.add_feature(cfeature.OCEAN)
ax.add_feature(cfeature.BORDERS)
ax.add_feature(cfeature.LAKES, alpha=0.5)
```

- **Example**: Plotting high-resolution San Francisco Bay area with 10m resolution coastlines and adding geographical features.

```python
ax.set_extent([-122.8, -122, 37.3, 38.3])
```

Animations in Python

- **Definition**:
  - An animation is a sequence of still frames displayed rapidly to create the illusion of motion.
  - Useful for better visual representation of dynamic systems compared to static plots.
- **Python Tools**:
  - The primary tool for creating animations in Python is the matplotlib.animation.Animation class.
  - This class offers a framework to build animation functionality.
- **Basic Steps for Creating an Animation**:

1. **Define Metadata**:
   - Define movie metadata such as title, artist, and comments using manimation.writers['ffmpeg'].
2. **Initialize Background**:
   - Set up static background elements like axis labels and curves.
3. **Update Frames**:
   - Inside a loop, update objects (like a moving point) for each frame.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# Data generation
n = 1000
x = np.linspace(0, 6 * np.pi, n)
y = np.sin(x)

# Create the figure and axis
fig, ax = plt.subplots()
ax.set_xlim(0, 6 * np.pi)
ax.set_ylim(-1.5, 1.5)
ax.set_xlabel('x')
ax.set_ylabel('sin(x)')

# Plot the sine wave (background)
sine_line, = ax.plot(x, y, 'b')

# Create an empty red circle that will move
red_circle, = ax.plot([], [], 'ro', markersize=10)

# Initialize the animation
def init():
    red_circle.set_data([], [])
    return red_circle,

# Update function for animation
def update(frame):
    red_circle.set_data([x[frame]], [y[frame]])
    return red_circle,
```

```
# Create the animation
ani = animation.FuncAnimation(
    fig,
    update,
    frames=n,
    init_func=init,
    interval=15,
    blit=True
)

plt.show()
```

Example: Sine Wave Animation

- **Objective**: Animate a red circle moving along a blue sine wave.
- **Key Components of Code**:
  - Import required libraries: numpy, matplotlib.pyplot, and matplotlib.animation.
  - Set up metadata for the movie (e.g., frames per second: fps=15).
  - **Sine Wave Setup**:
    - Use np.linspace() to generate 1000 data points for x, and np.sin(x) for the sine curve.
  - **Figure Initialization**:
    - Plot the sine wave and an empty placeholder for the red circle.
  - **Frame Updates**:
    - For each frame in a loop, update the position of the red circle along the sine wave.
- **Saving the Animation**:
  - Output the animation as an mp4 file with a resolution of 100 dpi using writer.saving().

```python
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as manimation

# Data generation
n = 1000
x = np.linspace(0, 6*np.pi, n)
y = np.sin(x)

# Define movie metadata
FFMpegWriter = manimation.writers['ffmpeg']
metadata = dict(title='Sine Wave Animation', artist='Matplotlib', comment='A red
writer = FFMpegWriter(fps=15, metadata=metadata)

# Initialize the movie
fig, ax = plt.subplots()
ax.set_xlim(0, 6*np.pi)
ax.set_ylim(-1.5, 1.5)
ax.set_xlabel('x')
ax.set_ylabel('sin(x)')

# Plot the sine wave
sine_line, = ax.plot(x, y, 'b')
red_circle, = ax.plot([], [], 'ro', markersize=10)

# Update the frames for the movie
with writer.saving(fig, "sine_wave_animation.mp4", 100):
    for i in range(n):
        red_circle.set_data(x[i], y[i])  # Update the red circle's position
        writer.grab_frame()  # Capture the frame

print("Animation saved as sine_wave_animation.mp4")
```